

**INTERNATIONAL CYBERSECURITY AND DIGITAL
FORENSICS ACADEMY (ICDFA)**

**INTRO TO CRITICAL INFRASTRUCTURE FOR IT
LEARNERS**

**LAB 06
INTRODUCTION TO MODBUS**

Student:	Adamu Nando Usufa
Registration No.:	C11/26/ACIS/17367
Course:	Intro to Critical Infrastructure for IT Learners
Lab:	Lab 06– Introduction to Modbus
Date:	October 2, 2026

1. Objective

The objective of this lab was to observe Modbus TCP communication between FUXA and OpenPLC in an isolated laboratory environment, capture the communication using Wireshark, and identify Modbus function codes, register information, and request/response fields.

The lab also provided practical understanding of Modbus TCP, Modbus RTU, and common Modbus function codes.

2. Laboratory Environment

The experiment was performed using isolated VirtualBox virtual machines.

2.1 Network Configuration Used

The actual network configuration used during this lab was:

Table 1: Laboratory Network Configuration

System	Interface	IP Address	Network / Purpose
PLC	enp0s3	192.168.1.10/24	ICDFA-Lab internal network
SCADA	enp0s8	192.168.1.20/24	ICDFA-Lab internal network
Kali / Analyst	eth0	192.168.1.30/24	ICDFA-Lab internal network
SCADA	enp0s3	10.0.2.15/24	NAT interface; not used for Modbus capture

The VirtualBox network used for the PLC/SCADA/Kali communication was the isolated Internal Network named **ICDFA-Lab**.

The Modbus communication observed during the exercise was:

$$SCADA/FUXA(192.168.1.20) \longrightarrow PLC(192.168.1.10)$$

using Modbus TCP over TCP port 502.

Wireshark captured the traffic through the Kali interface:

$$Kalieth0(192.168.1.30)$$

2.2 Connection Configuration

FUXA was configured as the Modbus TCP client and OpenPLC as the Modbus server. The FUXA connection targeted:

$$192.168.1.10:502$$

The HMI was running while the packet capture was performed.

3. FUXA/OpenPLC Process

The FUXA HMI was executed with the OpenPLC TankFillSim process running. The tank level changed during operation as seen in the three images captured during the processing below, confirming communication between the HMI and PLC.

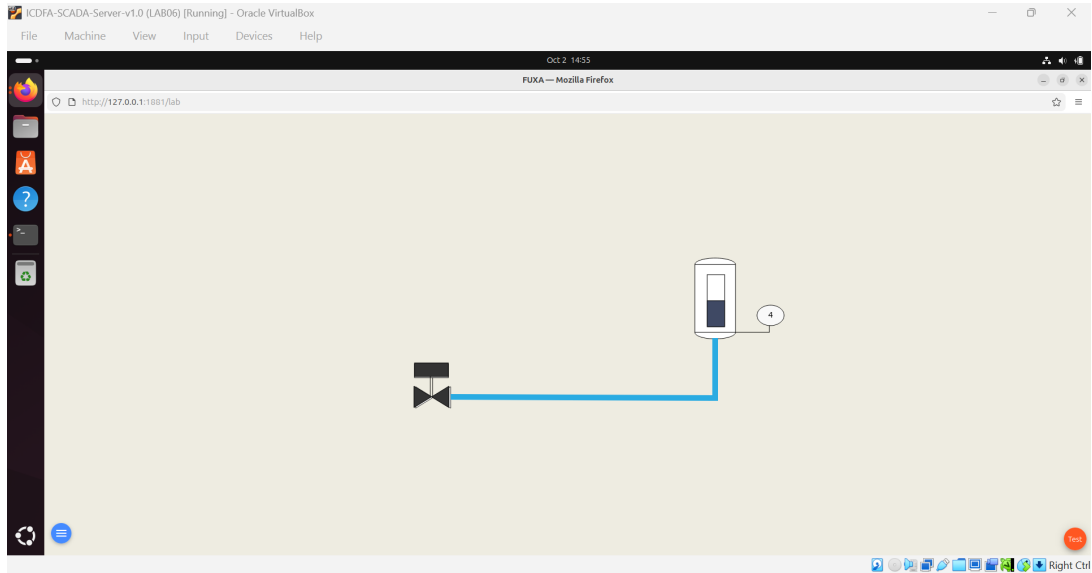


Figure 1: FUXA/OpenPLC tank-fill process running.

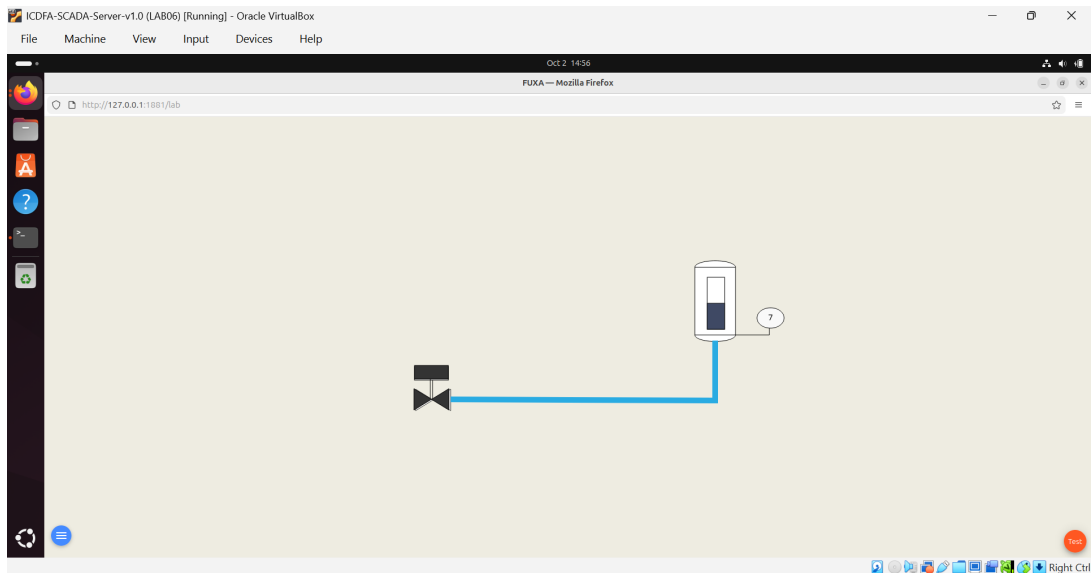


Figure 2: FUXA/OpenPLC tank-fill process running.

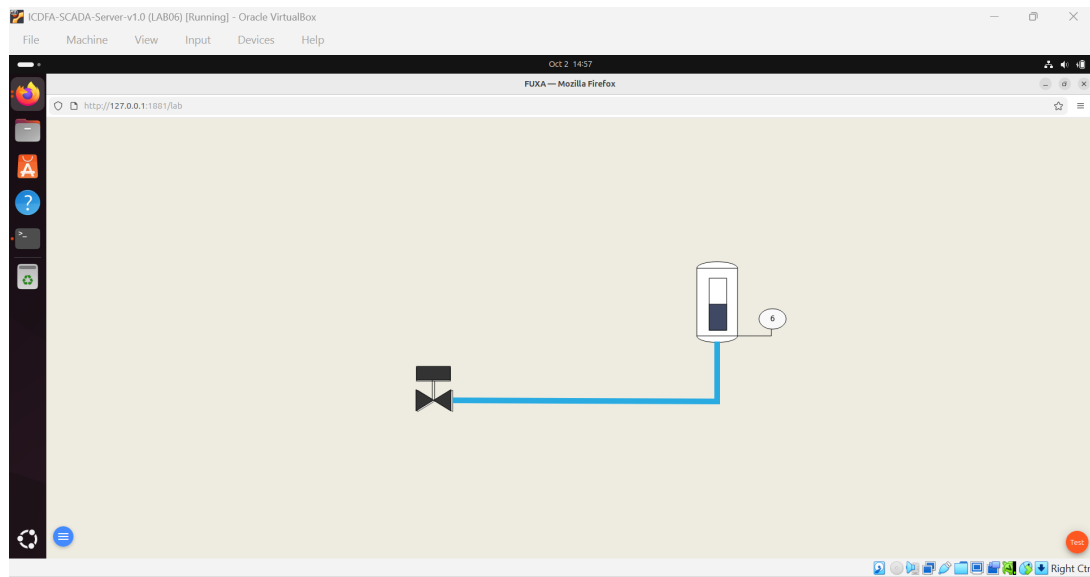


Figure 3: FUXA/OpenPLC tank-fill process running.

4. Wireshark Modbus Capture

Wireshark was configured on the Kali VM using the `eth0` interface. The display filter used was:

`modbus`

The capture showed Modbus traffic between:

- Source: 192.168.1.20 – SCADA
- Destination: 192.168.1.10 – PLC
- Protocol: Modbus TCP
- TCP Port: 502

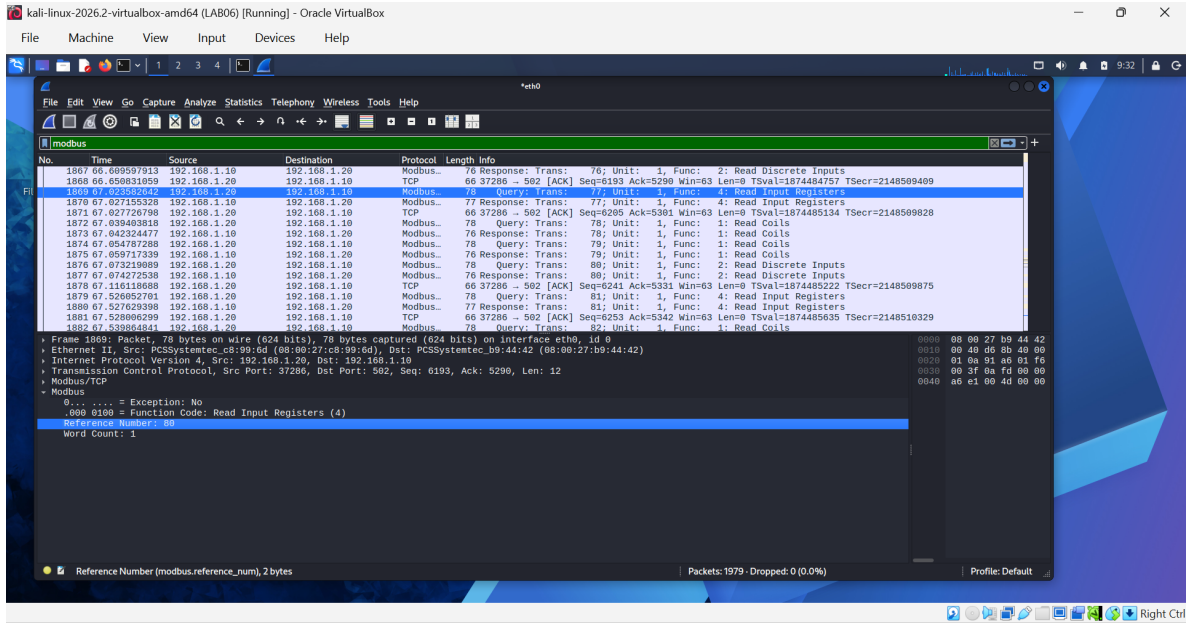


Figure 4: Wireshark showing Modbus TCP communication between SCADA and PLC.

5. Modbus Request/Response Analysis

A Modbus request/response pair was selected from the Wireshark capture.
The selected transaction was:

Table 2: Observed Modbus Function Code 4 Transaction

Field	Observed Value
Transaction ID	77
Source	192.168.1.20 (SCADA)
Destination	192.168.1.10 (PLC)
Protocol	Modbus TCP
TCP Port	502
Unit ID	1
Function Code	04 – Read Input Registers
Reference Number	80
Word Count	1
Purpose	Tank Level indication

The selected request was packet 1869 and the corresponding response was packet 1870.
The request showed:

Function Code 4 – Read Input Registers

with:

Reference Number = 80

In the teaching configuration, Input Register 80 corresponds to the Tank Level indication.

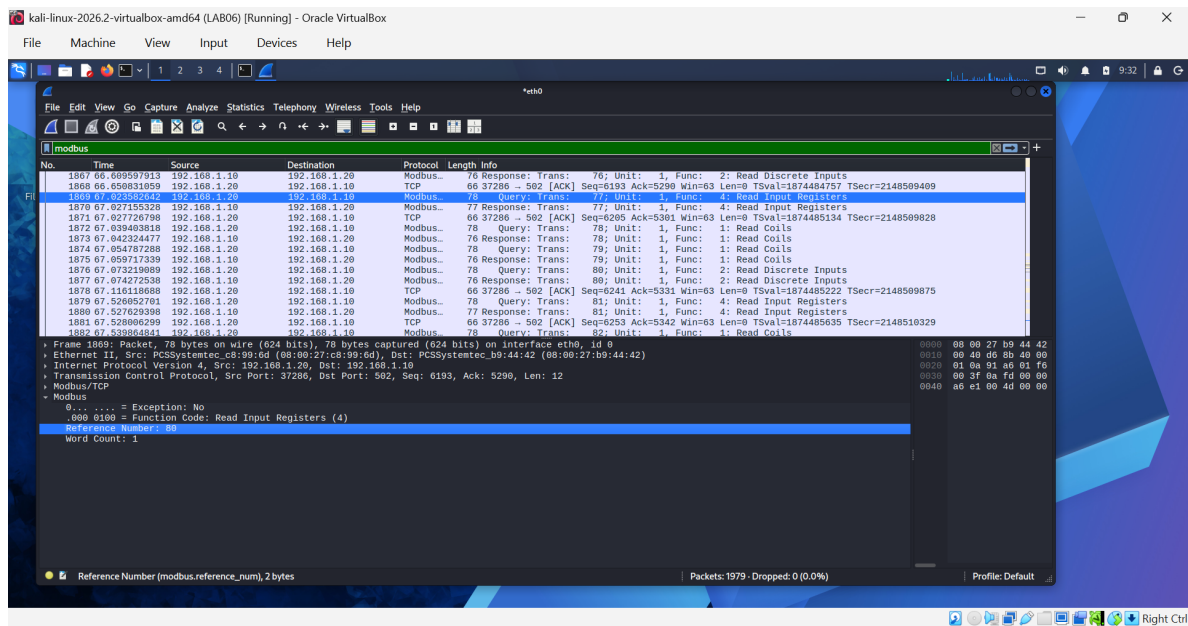


Figure 5: Wireshark showing Modbus TCP communication between SCADA and PLC.

6. Modbus Function-Code Table

The following table summarizes the required Modbus function codes.

Table 3: Common Modbus Function Codes

Code	Function	Purpose
01	Read Coils	Read discrete output/coil states.
02	Read Discrete Inputs	Read discrete input states.
03	Read Holding Registers	Read 16-bit holding registers.
04	Read Input Registers	Read 16-bit input registers. Used in this lab to read Tank Level Register 80.
05	Write Single Coil	Change one discrete coil/output state.
06	Write Single Holding Register	Change one 16-bit holding register.
15	Write Multiple Coils	Change multiple discrete coils/outputs.
16	Write Multiple Holding Registers	Change multiple holding registers.

7. Modbus TCP versus Modbus RTU

Table 4: Comparison of Modbus TCP and Modbus RTU

Feature	Modbus TCP	Modbus RTU
Transport	TCP/IP	Serial communication
Typical network	Ethernet	Serial bus such as RS-485
Communication	Client/server over TCP/IP	Serial master/slave communication
Port / addressing	TCP port 502	Serial communication parameters
Frame handling	Modbus message is carried over TCP/IP	Binary Modbus RTU frame with serial framing and CRC
Observed in this lab	Yes	No

Modbus TCP carries Modbus communication over TCP/IP, while Modbus RTU is used over serial communication. The Modbus specification defines the application-layer function codes independently of the underlying network technology.

8. Saved Packet Capture

The Wireshark capture was saved in .pcapng format.

The capture contains the Modbus traffic observed on the isolated ICDFA-Lab network.

Capture Details

- Capture interface: `eth0`
- Kali IP: `192.168.1.30`
- SCADA IP: `192.168.1.20`
- PLC IP: `192.168.1.10`
- Display filter: `modbus`
- Protocol: Modbus TCP
- TCP port: 502

9. Knowledge Check

9.1 1. What transport/networking difference separates Modbus TCP from Modbus RTU?

Response:

Modbus TCP uses TCP/IP over Ethernet, normally using TCP port 502. Modbus RTU uses serial communication, commonly over serial interfaces such as RS-485.

9.2 2. Why does Modbus use function codes?

Response:

Function codes identify the operation requested in a Modbus transaction, such as reading coils, reading registers, or writing registers. They allow the client and server to determine what operation should be performed.

9.3 3. What does Function Code 4 request/read?

Response:

Function Code 4 requests the reading of 16-bit input registers. In this lab, Function Code 4 was observed requesting Input Register 80, which corresponds to the Tank Level indication.

9.4 4. Why is visibility of unprotected industrial protocol traffic a cybersecurity concern?

Response:

Unprotected Modbus traffic can expose operational information such as device addresses, function codes, register references, and process data to someone who can observe the network traffic. This can assist unauthorized monitoring and, where write operations are possible, increase the risk of unauthorized control. Modbus TCP itself does not provide modern cryptographic protection; the Modbus Organization specification separately defines Modbus Security using TLS for protected communications.

10. Required Evidence Checklist

Table 5: Lab 06 Evidence Checklist

No.	Required Evidence	Status
1	FUXA/OpenPLC process running screenshot	Completed
2	Wireshark screenshot filtered to Modbus traffic	Completed
3	Annotated request/response identifying Function Code 4 and Register 80	Completed
4	Saved .pcapng capture from the isolated training network	Completed
5	Completed Modbus function-code table	Completed
6	Modbus TCP versus Modbus RTU comparison	Completed
7	Knowledge check responses	Completed

11. Conclusion

The lab demonstrated Modbus TCP communication between FUXA/SCADA and OpenPLC in an isolated virtual laboratory network. Wireshark was used to capture and filter the communication on the Kali analyst VM.

A Function Code 4 transaction was identified between the SCADA server (192.168.1.20) and PLC (192.168.1.10). The captured request referenced Input Register 80, corresponding to the Tank Level indication in the teaching configuration.

The exercise provided practical experience in identifying Modbus function codes, registers, request/response communication, and the difference between Modbus TCP and Modbus RTU.